PDS TECH

DEVOPS E ARQUITETURA CLOUD

FASE 4 | AULA 04 -

DATADOG - ALERTAS CUSTOMIZADOS, DASHBOARDS E ERRORS

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

SUMÁRIO

O QUE VEM POR AÍ? 3
HANDS ON 4
SAIBA MAIS 5
O QUE VOCÊ VIU NESTA AULA? 38
REFERÊNCIAS 39

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

O QUE VEM POR AÍ?

Nesta aula, o(a) aluno(a) aprofundará o entendimento sobre como **transformar dados de observabilidade em decisões acionáveis**, indo além da simples coleta de métricas. Serão explorados os princípios de **design eficaz de dashboards**, a correta escolha de visualizações e o uso de contexto para acelerar análises de incidentes. Em paralelo, o(a) estudante será introduzido ao **framework de SLOs e Error Budget**, compreendendo como medir confiabilidade sob a ótica do usuário e como utilizar conceitos como **burn rate** para orientar alertas inteligentes e priorização técnica. Ao final, o(a) aluno(a) passará a enxergar observabilidade como um instrumento estratégico de confiabilidade e não apenas como monitoramento operacional.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

HANDS ON

Durante os laboratórios práticos desta aula, você irá criar dashboards no Datadog seguindo princípios de hierarquia visual e padrão Z, e configurar widgets adequados para diferentes tipos de análise (Timeseries, Top List, Heatmap e Query Value).

Além disso, vamos construir dashboards baseados nos Golden Signals (latência, tráfego, erros e saturação) e aplicar correlação visual entre métricas de aplicação e infraestrutura. Vamos ainda adicionar Event Overlays para contextualizar métricas com deploys e eventos operacionais e definir SLIs e SLOs para uma aplicação .NET monitorada via APM.

Por fim, vamos configurar Error Budget e acompanhar seu consumo no Datadog, criar alertas baseados em burn rate (fast burn e slow burn) e comparar alertas baseados em causas vs. alertas baseados em sintomas.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

SAIBA MAIS

Princípios de design de dashboards eficazes

A construção de dashboards de monitoramento transcende a mera agregação de gráficos e métricas; trata-se de uma disciplina que combina ciência de dados, design de informação e um profundo entendimento do sistema em observação.

Um dashboard eficaz não é apenas um repositório de dados, mas uma ferramenta de comunicação estratégica que permite a equipes de engenharia, operações e negócios tomar decisões rápidas e informadas. No contexto de sistemas complexos e distribuídos, como os desenvolvidos na stack .NET, a capacidade de um dashboard de traduzir um volume massivo de telemetria em insights acionáveis é fundamental para a manutenção da saúde, performance e confiabilidade das aplicações.

Esta aula explorará os princípios fundamentais que norteiam o design de dashboards de alta performance no Datadog. Abordaremos como a aplicação de conceitos de hierarquia visual, densidade de informação, consistência e contextualização pode transformar um painel de dados reativo em um centro de comando proativo.

Dashboards em um ambiente de observabilidade cumprem três funções primordiais:

Monitoramento em Tempo Real: oferecer uma visão instantânea do estado atual do sistema, destacando a saúde dos serviços, a experiência do usuário e o cumprimento dos Níveis de Serviço (SLOs).

Análise e Investigação (Troubleshooting): servir como ponto de partida para investigações de incidentes, permitindo a correlação entre diferentes fontes de dados (métricas, traces, logs) para identificar a causa raiz de falhas.

Comunicação e Alinhamento: prover uma fonte única de verdade para diferentes stakeholders (desenvolvedores, SREs, gerentes de produto, executivos), alinhando todos sobre a performance e o impacto no negócio.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

O Padrão Z e Hierarquia Visual

A eficácia de um dashboard de monitoramento não reside apenas na precisão de seus dados, mas na velocidade com que um observador pode extrair insights e tomar ações. A hierarquia visual é o princípio de design que organiza e apresenta os elementos de informação de forma a comunicar sua ordem de importância. Em um cenário de incidentes, onde cada segundo conta, um dashboard bem estruturado guia o olhar do engenheiro através de um caminho lógico, da identificação do sintoma à análise da causa.

Em culturas ocidentais, a leitura de conteúdo digital tende a seguir padrões previsíveis. Um dos mais comuns é o Padrão Z (Z-Pattern). O olho do usuário primeiro varre o topo da página da esquerda para a direita (a barra superior do "Z"), depois se move para baixo e para a esquerda em uma linha diagonal, e finalmente varre a parte inferior da página da esquerda para a direita novamente (a barra inferior do "Z").

Este padrão de leitura natural deve ser explorado no design de dashboards. A informação mais crítica e de mais alto nível deve ser posicionada nos pontos de maior atenção do Padrão Z: o canto superior esquerdo e ao longo da borda superior. Informações secundárias e de suporte podem ser organizadas ao longo da diagonal, e os dados mais granulares e detalhados, na base do dashboard.

Complementar ao Padrão Z, o modelo da **Pirâmide Invertida** estrutura o conteúdo do dashboard em camadas de urgência e detalhe, de cima para baixo:

Topo (Visão Geral): esta é a camada mais importante e deve responder à pergunta: "O serviço está saudável?". É aqui que os **Quatro Golden Signals** (Latência, Tráfego, Erros e Saturação) devem residir. Esses indicadores de alto nível fornecem um resumo conciso e universal da saúde de qualquer serviço.

Meio (Tendências e Contexto): esta camada fornece o contexto para os sinais da camada superior. Gráficos de séries temporais com janelas de tempo mais longas, comparações com períodos anteriores (semana sobre semana) e desagregações por componentes (por exemplo, erros por endpoint) ajudam a entender o "porquê" por trás de uma anomalia.

Base (Detalhes e Análise Profunda): a camada inferior oferece os dados mais granulares para a investigação de causa raiz. Isso inclui streams de logs, listas de traces distribuídos, perfis de CPU e links para documentação ou runbooks.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Os **Golden Signals**, definidos pelo **Google SRE**, são um conjunto enxuto de métricas essenciais para avaliar a **saúde real de sistemas distribuídos sob a ótica do usuário**. Em vez de monitorar dezenas de indicadores isolados, o conceito propõe focar no que realmente importa para a **experiência**, **confiabilidade** e **capacidade operacional** do serviço.

Golden Signal	Descrição	Métrica Datadog para .NET (Exemplo)
Latência	O tempo necessário para atender a uma requisição. É crucial distinguir a latência de requisições bem-sucedidas e falhas.	`trace.aspnetcore.request.duration` (p95, p99)
Tráfego	A quantidade de demanda que está sendo colocada no sistema, medida em uma métrica de alto nível específica do sistema.	`trace.aspnetcore.request.hits` (requisições por segundo)
Erros	A taxa de requisições que falham, seja explicitamente (ex: HTTP 5xx) ou implicitamente (ex: HTTP 200 com conteúdo incorreto).	`trace.aspnetcore.request.errors`
Saturação	Quão "cheio" está o seu serviço, medindo os recursos mais restritos (CPU, memória, I/O). A latência é frequentemente um indicador antecedente da saturação.	`system.cpu.user`, `process.mem.rss`

Figura 1 – Golden Signals
Fonte: Elaborado pelo autor (2026)

Densidade e Correlação Visual

A densidade de informação em um dashboard refere-se à quantidade de dados e visualizações apresentadas em uma única tela. O desafio do design eficaz é encontrar o ponto de equilíbrio: um dashboard muito esparso pode esconder informações cruciais, exigindo que o usuário navegue por múltiplas telas para obter uma visão completa. Por outro lado, um dashboard excessivamente denso pode levar à sobrecarga cognitiva, tornando difícil para o cérebro processar a informação e identificar os sinais importantes em meio ao ruído.

O objetivo não é maximizar a quantidade de dados, mas sim a eficiência da transferência de informação. Cada widget em um dashboard deve ter um propósito claro e justificar o espaço que ocupa. Uma boa prática é seguir o princípio de "visão geral primeiro, zoom e filtro, depois detalhes sob demanda".

No Datadog, isso pode ser alcançado de várias maneiras:

Dashboards de Níveis Múltiplos: crie um dashboard de alto nível (Nível 1) com os Golden Signals e os KPIs mais críticos. A partir deste, crie links para dashboards de Nível 2, que fornecem uma visão mais detalhada de um serviço ou

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

subsistema específico (por exemplo, um dashboard focado apenas no banco de dados ou em um microsserviço específico).

Template Variables: utilize variáveis de template para permitir que os usuários filtrem dinamicamente o dashboard para um contexto específico (por exemplo, por host, region, availability-zone ou customer-id). Isso permite que um único dashboard sirva a múltiplos propósitos sem sobrecarregar a visão padrão.

Poder dos Grupos (Powerpacks): agrupe widgets relacionados em seções colapsáveis usando o widget de Grupo ou Powerpack. Isso mantém o dashboard organizado e permite que os usuários expandam apenas as seções relevantes para sua investigação atual.

O cérebro humano é programado para encontrar padrões. A **correlação visual** aproveita essa capacidade inata, posicionando gráficos e métricas relacionados próximos uns dos outros para facilitar a identificação de causalidade e correlação. Quando um incidente ocorre, a capacidade de ver como uma métrica afeta outra em tempo real é inestimável.

Um dos exemplos mais poderosos de correlação visual é a sobreposição de métricas de diferentes fontes em um único gráfico de série temporal. Isso permite que os engenheiros validem hipóteses rapidamente. Por exemplo: "O pico de latência na nossa API .NET está correlacionado com um aumento na utilização da CPU do nosso banco de dados SQL Server? Princípios de Proximidade para Correlação:

Agrupamento Lógico: coloque widgets que representam partes do mesmo subsistema juntos. Por exemplo, métricas de um serviço de API devem estar próximas das métricas do banco de dados que ele consome.

Eixos de Tempo Sincronizados: ao usar múltiplos gráficos de série temporal, garanta que seus eixos de tempo estejam sincronizados. O Datadog faz isso por padrão, mas é crucial manter essa consistência. Quando um usuário dá zoom em um período de tempo em um gráfico, todos os outros gráficos devem refletir o mesmo período.

Sobreposição de Métricas: não hesite em sobrepor métricas de diferentes fontes em um único gráfico para destacar relações diretas. Por exemplo, sobrepor a taxa de erros de uma aplicação com a contagem de deploys pode revelar imediatamente se uma nova versão introduziu problemas.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Consistência visual

A consistência visual é a base para a usabilidade e a interpretabilidade de qualquer interface de dados. Em um ambiente de alta pressão como o monitoramento de aplicações, onde a velocidade de compreensão é crítica, um design consistente reduz a carga cognitiva do usuário. Quando cores, eixos e padrões de interação se comportam de maneira previsível, os engenheiros podem parar de decifrar o *dashboard* e começar a decifrar o *problema*. A consistência cria uma linguagem visual intuitiva, permitindo que o conhecimento adquirido em um gráfico seja imediatamente aplicado a outro.

Em dashboards de monitoramento, a cor nunca deve ser meramente decorativa; ela é um poderoso canal de informação. O uso consistente de cores para representar significados específicos cria atalhos mentais que aceleram a detecção de problemas.

Melhores Práticas para o Uso de Cores:

Paleta Limitada e Estável: defina uma paleta de cores padrão para toda a sua organização. Limite o número de cores distintas para evitar poluição visual. Ferramentas como o colorbrewer podem ajudar a escolher paletas que são visualmente distintas e acessíveis.

Significados Fixos: atribua significados consistentes às cores. O padrão mais comum é:

- **Vermelho:** alerta crítico, erro, perigo, violação de SLO.
- **Amarelo/Laranja:** aviso, anomalia, saturação se aproximando do limite.
- **Verde:** saudável, OK, sucesso.
- **Azul:** informacional, tráfego, métricas neutras.

Acessibilidade: lembre-se que uma parcela da população possui alguma forma de daltonismo. A cor não deve ser o *único* meio de transmitir informação. Use-a em conjunto com ícones, rótulos de texto ou padrões para garantir que todos possam interpretar o estado do sistema.

Os eixos de um gráfico fornecem a escala e o contexto para os dados. A inconsistência nos eixos, especialmente no eixo Y (valor), pode levar a interpretações

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

drasticamente equivocadas. Se dois gráficos lado a lado mostram a mesma métrica para dois serviços diferentes, mas seus eixos Y têm escalas automáticas e diferentes, uma pequena flutuação em um serviço pode parecer visualmente maior do que um grande pico em outro.

Melhores Práticas para Eixos:

1. **Eixos Y Fixos para Comparação:** ao comparar a mesma métrica entre diferentes escopos (por exemplo, uso de CPU em múltiplos hosts), fixe o eixo Y em uma faixa lógica e consistente (por exemplo, 0 a 100%). Isso fornece uma base de comparação verdadeira.
2. **Sincronização do Eixo X:** o Datadog sincroniza o eixo de tempo (eixo X) em todos os widgets de um dashboard por padrão. Isso é fundamental para a correlação visual e deve ser mantido.
3. **Uso Criterioso de Eixos Duplos:** eixos Y duplos são úteis para correlacionar métricas com unidades ou escalas muito diferentes (como latência em ms e CPU em %). No entanto, use-os com cuidado, pois podem ser confusos. Sempre rotule claramente qual linha corresponde a qual eixo.

A consistência deve se estender além da aparência para o comportamento do dashboard. A forma como um usuário interage com os dados deve ser previsível.

Contexto é rei: Event Overlays e Markers

Um gráfico de série temporal que exibe um pico de latência é uma observação. O mesmo gráfico que mostra que o pico de latência começou exatamente um minuto após um evento de deploy é uma narrativa—uma narrativa que aponta diretamente para uma possível causa raiz. No universo da observabilidade, contexto é o que transforma dados em insights acionáveis.

O Datadog eleva o design de dashboards ao permitir que esse contexto seja visualmente integrado diretamente sobre os dados. As ferramentas para isso são os **Event Overlays** e os **Markers**. Elas são projetadas para responder à pergunta mais importante durante uma investigação: “O que mais aconteceu neste exato momento?”

Event Overlays são marcadores verticais que aparecem em gráficos de série temporal para indicar a ocorrência de eventos discretos. Eles são a ponte visual entre

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

as métricas contínuas (como latência ou uso de CPU) e os eventos pontuais que podem influenciá-las.

Quando um evento é enviado para o Datadog com as tags corretas (por exemplo, service, env, host), ele aparece automaticamente em todos os gráficos que estão escopados com as mesmas tags. Isso cria uma correlação passiva e poderosa, enriquecendo cada visualização com um histórico de atividades relevantes.

Fontes Comuns de Eventos:

- **Deploys e Alterações de Código:** a fonte mais crítica de contexto. Integrar seu pipeline de CI/CD (Azure DevOps, GitHub Actions, Jenkins) para enviar um evento ao Datadog a cada deploy bem-sucedido é uma prática fundamental. O evento deve conter tags com a versão do código, o serviço e o ambiente.
- **Alertas de Monitores:** quando um monitor do Datadog dispara um alerta, ele pode gerar um evento. Isso é útil para visualizar falhas em cascata, onde o alerta de um serviço dependente (como um banco de dados) precede um problema no serviço principal.
- **Alterações de Infraestrutura:** ferramentas de gerenciamento de configuração como Ansible, Chef ou Terraform podem ser configuradas para enviar eventos quando aplicam alterações em um ambiente.
- **Eventos de Negócio:** lançamento de uma campanha de marketing, início de uma promoção de vendas, ou o acionamento de uma feature flag. Esses eventos, quando correlacionados com métricas de tráfego e performance, fornecem insights valiosos sobre o comportamento do sistema sob diferentes condições de negócio.

Se os Event Overlays marcam o *quando*, os **Markers** (marcadores) definem o *o quê* e o *quanto*. Eles são linhas ou áreas sombreadas horizontais que adicionam limiares e zonas de referência aos gráficos, fornecendo um contexto de magnitude instantâneo.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Tipos de Markers e Seus Usos

Linhas de Alerta (Alert Lines): uma linha sólida que representa o limiar de um monitor. Isso mostra visualmente o quão perto uma métrica está de disparar um alerta.

Zonas de Perigo (Danger Zones): uma área sombreada que indica uma faixa de valores indesejável. Por exemplo, em um gráfico de uso de disco, uma área sombreada de vermelho acima de 90% indica claramente a zona de perigo.

Limiares de SLO (SLO Thresholds): a aplicação mais importante. Em um gráfico de latência ou taxa de sucesso, uma linha horizontal representando o alvo do SLO (por exemplo, 200ms de latência ou 99.9% de sucesso) comunica imediatamente se o serviço está cumprindo suas promessas de confiabilidade.

Baselines Históricas: uma linha pontilhada mostrando o valor da métrica no mesmo período anterior (por exemplo, na semana passada), útil para identificar desvios do comportamento normal.

Escolhendo o widget certo

O Datadog oferece um arsenal de widgets de visualização, cada um projetado para responder a um tipo diferente de pergunta sobre os dados. A escolha do widget certo é tão crucial quanto a escolha da métrica certa. Um widget inadequado pode obscurecer insights, enquanto o widget correto pode revelá-los de forma clara e imediata. A arte do design de dashboards reside em mapear a pergunta que você quer responder para a visualização mais eficaz.

Timeseries (Série Temporal): exibe a evolução de uma ou mais métricas ao longo do tempo.

- **Pergunta que Responde:** “Como essa métrica mudou ao longo do tempo?” ou “Existe uma correlação temporal entre essas métricas?”
- **Pontos Fortes:** excelente para identificar tendências, padrões sazonais, picos, quedas e para correlacionar o comportamento de diferentes métricas (como latência vs. CPU).
- **Caso de Uso:** visualizar a latência p95 da sua API ASP.NET Core, a taxa de erros e o número de requisições por segundo nas últimas 4 horas para entender o comportamento recente do serviço.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Exemplo de Query:

- p95:trace.aspnetcore.request.duration{service:minha-api-net}, sum:trace.aspnetcore.request.errors{service:minha-api-net}.as_count()

Top List (Lista dos Principais): ranqueia um conjunto de itens (como hosts, containers, endpoints) com base no valor de uma métrica.

- **Pergunta que Responde:** “Quais são os X que mais contribuem para esta métrica?” ou “Onde devo focar minha atenção?”
- **Pontos Fortes:** perfeito para identificar os maiores ofensores ou os itens mais impactados. Direciona a investigação para os elementos mais relevantes, economizando tempo.
- **Caso de Uso para .NET:** identificar os 10 endpoints de API (resource_name) com a maior latência média, ou os clientes (customer_id) que estão gerando o maior número de erros 500.

Exemplo de Query:

- top(avg:trace.aspnetcore.request.duration{service:minha-api-net} by {resource_name}, 10, 'max', 'desc')

Heatmap (Mapa de Calor): visualização bidimensional que usa a cor para representar a magnitude de uma métrica em relação a duas variáveis.

- **Pergunta que Responde:** “Qual é a distribuição da latência para todas as minhas requisições, não apenas um percentil?” ou “A maioria das minhas requisições é rápida, com algumas poucas muito lentas, ou a latência está uniformemente distribuída?”
- **Pontos Fortes:** revela a distribuição completa de uma métrica, ao contrário de agregadores como média ou p99, que podem esconder a complexidade dos dados. É excelente para identificar outliers e entender a experiência de todos os usuários, não apenas da maioria.
- **Caso de Uso:** visualizar a distribuição da latência de resposta (trace.aspnetcore.request.duration) ao longo da última hora. As cores mais quentes (mais escuras) indicam as faixas de latência onde a maioria das requisições se concentra, enquanto pontos de cor isolados em faixas de

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

latência mais altas revelam outliers que seriam invisíveis em um gráfico de média.

Exemplo de Query:

- avg:trace.aspnetcore.request.duration{service:minha-api-net} by {host}.

Query Value (Valor da Consulta): exibe o valor agregado de uma métrica em um único número grande e proeminente. É um widget de status, projetado para ser lido em uma fração de segundo.

- **Pergunta que Responde:** “Qual é o estado desta métrica *agora*?”
- **Pontos Fortes:** Ideal para KPIs (Key Performance Indicators) e os Golden Signals. Sua principal força é a formatação condicional, que permite que o fundo do widget mude de cor (verde, amarelo, vermelho) com base em limiares pré-definidos, fornecendo um sinal de saúde instantâneo.
- **Caso de Uso:** exibir a taxa de sucesso geral da aplicação ((1 - sum:trace.aspnetcore.request.errors / sum:trace.aspnetcore.request.hits) * 100). O fundo fica verde se for > 99.9% (SLO atingido), amarelo se estiver entre 99.5% e 99.9%, e vermelho se for < 99.5%.

Exemplo de Query:

- (1 - sum:trace.aspnetcore.request.errors{service:minha-api-net}.as_count() / sum:trace.aspnetcore.request.hits{service:minha-api-net}.as_count()) * 100

Framework de SLOs e Error Budget

Subtopico 2.1 - SLI, SLO e Error Budget

No cenário moderno de engenharia de software, a discussão sobre a performance e a disponibilidade de um serviço evoluiu de uma abordagem puramente reativa, baseada em alertas de falhas, para uma metodologia proativa e centrada no usuário, fundamentada em confiabilidade quantificada.

O framework de Objetivos de Nível de Serviço (SLOs) e Orçamentos de Erro (Error Budgets), popularizado pela engenharia de confiabilidade de sites (SRE) do Google, representa o ápice dessa evolução. Ele fornece uma linguagem comum e

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

baseada em dados para que equipes de produto, desenvolvimento e operações possam tomar decisões estratégicas sobre o equilíbrio entre a inovação (lançamento de novas features) e a confiabilidade (garantia da estabilidade do serviço).

Buscar 100% de confiabilidade não é apenas impossível, mas também um mau negócio. O custo e o esforço para eliminar a última fração de falha são exponenciais e, mais importante, os usuários geralmente não percebem a diferença entre 99.99% e 100% de disponibilidade. O framework de SLOs formaliza essa noção, permitindo que as equipes definam um nível de confiabilidade *alvo* que seja bom o suficiente para manter os usuários felizes e, ao mesmo tempo, permitir um ritmo saudável de desenvolvimento.

Compreender a distinção e a relação entre o Indicador de Nível de Serviço (SLI), o Objetivo de Nível de Serviço (SLO) e o Orçamento de Erro (Error Budget) é o primeiro passo para transformar a confiabilidade de uma meta abstrata em uma prática de engenharia mensurável e gerenciável.

O **SLI** é uma medida quantitativa de algum aspecto do nível de serviço que é fornecido. É a evidência crua e direta da performance do seu serviço do ponto de vista do usuário. Um SLI não é apenas uma métrica; é uma métrica formatada como uma proporção de "eventos bons" em relação ao "total de eventos válidos".

Fórmula Geral: SLI = (Eventos Bons / Total de Eventos Válidos) * 100

- **Eventos Bons:** Interações que atenderam aos critérios de sucesso (ex: requisição retornou HTTP 200, latência foi inferior a 300ms).
- **Total de Eventos Válidos:** Todas as interações legítimas que deveriam ter sido bem-sucedidas. É importante excluir eventos inválidos (ex: requisições com erro de cliente, como HTTP 400).

O **SLO** é o objetivo ou alvo para o valor de um SLI ao longo de um período de tempo. É um acordo interno que define o que a equipe considera um nível de serviço "bom o suficiente" para manter os usuários satisfeitos. Um SLO é sempre composto por três partes: o SLI, um alvo percentual e uma janela de tempo.

Estrutura de um SLO: [SLI] >= [Alvo Percentual] durante [Janela de Tempo]

Exemplo: "A proporção de requisições bem-sucedidas na API de pagamentos será maior ou igual a 99.9% nos últimos 30 dias."

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

O **Error Budget** é a consequência direta e a ferramenta mais poderosa que deriva de um SLO. Ele representa a quantidade de tempo ou o número de eventos em que o serviço pode falhar sem violar o SLO. É simplesmente 100% - SLO%.

Fórmula do Error Budget: Error Budget = 1 - (SLO / 100)

Este orçamento transforma a discussão sobre confiabilidade. Em vez de buscar uma perfeição inatingível, as equipes agora gerenciam um orçamento. Se o orçamento está sendo gasto rapidamente, é hora de focar em estabilidade. Se há orçamento sobrando, a equipe tem a liberdade para inovar e assumir riscos calculados.

Conceito	Pergunta que Responde	Exemplo	Tipo
SLI	O que estamos medindo?	Proporção de requisições com latência < 300ms.	Medida
SLO	Quão bom nosso serviço precisa ser?	99.5% das requisições devem ter latência < 300ms em 28 dias.	Alvo
Error Budget	Quanto espaço temos para falhar?	0.5% das requisições podem ter latência > 300ms em 28 dias.	Orçamento

Figura 2 – Error Budget
Fonte: Elaborado pelo autor (2026)

Alertas baseados em sintomas vs causas

Uma das mudanças de paradigma mais significativas introduzidas pelo framework de SLOs é a redefinição do que constitui um alerta acionável. Tradicionalmente, os sistemas de monitoramento eram configurados com uma infinidade de alertas baseados em causas. Estes são alertas que disparam quando uma métrica de baixo nível de um componente do sistema cruza um limiar estático. O exemplo clássico e mais notório é o alerta de "Uso de CPU > 90%".

O problema fundamental com os alertas baseados em causas é que eles frequentemente não têm uma correlação direta com a experiência do usuário. Um servidor pode operar com 95% de CPU e ainda assim atender a todas as requisições dentro do SLO de latência. Nesses casos, um alerta de CPU alta é apenas ruído. Ele acorda um engenheiro no meio da noite para "resolver" um problema que não está, de fato, impactando os usuários. Essa fadiga de alertas é perigosa, pois leva as equipes a ignorar notificações que podem, eventualmente, ser importantes.

Em contrapartida, os alertas baseados em sintomas focam no que realmente importa: o impacto no usuário. Eles disparam quando a experiência do usuário está

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

se degradando ou em risco iminente de degradação. Esses alertas estão intrinsecamente ligados aos seus SLIs e SLOs.

Característica	Alertas Baseados em Causas (Ruim)	Alertas Baseados em Sintomas (Bom)
O que mede?	Métricas de componentes internos do sistema.	Métricas que refletem a experiência do usuário.
Foco	A saúde de uma máquina ou processo.	A saúde do serviço como um todo.
Exemplo	`CPU do host > 90% por 5 minutos.`	`A latência p99 das requisições > 300ms por 10 minutos.`
Acionabilidade	Baixa. Frequentemente não requer ação imediata.	Alta. Indica um impacto real no usuário que precisa ser investigado.
Correlação com o Negócio	Indireta e incerta.	Direta. A violação do sintoma afeta a satisfação do cliente.
Fonte da Verdade	Métricas de infraestrutura (ex: `system.cpu.user`).	SLIs (ex: `trace.aspnetcore.request.duration`).

Figura 3 – Alertas
Fonte: Elaborado pelo autor (2026)

Isso não significa que os alertas baseados em causas devam ser completamente eliminados. Eles ainda têm um lugar, mas não como alertas que exigem uma intervenção humana imediata (ou seja, que paginam um engenheiro). Em vez disso, eles devem ser usados para **notificações de baixa prioridade** ou para **automação**.

Notificações (Tickets/Slack): um alerta de "Uso de disco > 80%" não precisa acordar ninguém, mas pode criar automaticamente um ticket de baixa prioridade para a equipe de infraestrutura investigar durante o horário comercial. Isso se torna um trabalho planejado, não uma emergência.

Automação (Auto-remediação): um alerta de "Número de pods de uma aplicação .NET abaixo do mínimo" pode acionar um script automatizado para reiniciar o deployment no Kubernetes, sem qualquer intervenção humana.

Cálculo real de Error Budget e limites

O conceito de Error Budget (Orçamento de Erro) pode parecer abstrato quando expresso apenas como uma porcentagem (por exemplo, 0.1%). Sua verdadeira força como ferramenta de engenharia e gerenciamento de produtos se revela quando o traduzimos em unidades concretas e tangíveis: minutos de indisponibilidade permitidos ou número de requisições falhas permitidas.

Essa tradução transforma o SLO de um número em um contrato claro sobre a quantidade de imperfeição que o serviço pode apresentar antes de impactar negativamente a confiança do usuário e os objetivos de negócio.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

O cálculo do Error Budget depende fundamentalmente do tipo de SLI que está sendo medido: baseado em tempo (time-based) ou baseado em eventos (event-based).

- **SLIs Baseados em Tempo**: geralmente usados para medir disponibilidade (uptime). O orçamento de erro é calculado como uma fração da janela de tempo total do SLO.
- **SLIs Baseados em Eventos**: usados para medir a qualidade de interações discretas, como latência de requisições ou taxa de sucesso. O orçamento de erro é calculado como uma fração do número total de eventos na janela de tempo.

Uma forma comum de expressar a disponibilidade alvo é através do número de "noves" na porcentagem do SLO. Compreender o que esses noves significam em termos de tempo de inatividade (downtime) permitido é crucial para definir SLOs realistas.

Nível do SLO (%)	"Noves"	Downtime Permitido por Dia	Downtime Permitido por Semana (7 dias)	Downtime Permitido por Mês (30 dias)
99.0%	2 Noves	14m 24s	1h 40m 48s	7h 18m 17s
99.5%	-	7m 12s	50m 24s	3h 39m 8s
99.9%	3 Noves	1m 26s	10m 4s	**43m 49s**
99.95%	-	43s	5m 2s	21m 54s
99.99%	4 Noves	8.6s	1m 0s	4m 21s
99.999%	5 Noves	0.86s	6s	26s

Figura 4 – SLOs
Fonte: Elaborado pelo autor (2026)

Medindo a velocidade de falha (Burn Rate)

Uma vez que um Error Budget é estabelecido, a próxima pergunta lógica é: "Com que rapidez estamos consumindo nosso orçamento?" O Burn Rate (taxa de queima) é a métrica que responde a essa pergunta. Ele normaliza a taxa de consumo do Error Budget, fornecendo um indicador poderoso sobre a gravidade de um problema e o tempo estimado até o esgotamento completo do orçamento.

O Burn Rate é um multiplicador. Um burn rate de 1 significa que você está consumindo o orçamento exatamente na taxa esperada para que ele se esgote no final da janela do SLO (por exemplo, em 30 dias). Um burn rate de 2 significa que você

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

está consumindo o orçamento duas vezes mais rápido e, se mantido, ele se esgotará na metade da janela do SLO (em 15 dias). Um burn rate de 10 indica que o orçamento se esgotará em um décimo do tempo (em 3 dias).

Fórmula: Burn Rate = (Taxa de Erro Observada / Taxa de Erro do Orçamento)

Onde a Taxa de Erro do Orçamento é 1 - (SLO Target / 100).

O poder do Burn Rate reside em sua capacidade de contextualizar a taxa de erro. Uma taxa de erro de 1% pode ser aceitável para um SLO de 95%, mas catastrófica para um SLO de 99.99%. O Burn Rate unifica essa análise, fornecendo um indicador comparável entre diferentes SLOs.

Nem todas as falhas são criadas iguais. Um incidente catastrófico que derruba completamente o serviço requer uma resposta imediata, enquanto uma pequena degradação de performance pode ser tratada com menos urgência. Para capturar essa nuance, a prática recomendada é configurar dois tipos de alertas baseados em burn rate para cada SLO: um para **burn rápido** e outro para **burn lento**.

Burn Rate Rápido (Fast Burn)

- **Objetivo:** detectar incidentes graves e de alto impacto que consomem o Error Budget muito rapidamente e ameaçam violar o SLO em questão de horas ou poucos dias.

Características:

- **Limiar de Burn Rate:** alto (ex: 10x, 15x, 20x).
- **Janela de Avaliação:** curta (ex: 5 a 60 minutos).

Lógica: "Se estamos queimando nosso orçamento de 30 dias a uma taxa que o esgotaria em 2 dias, e isso está acontecendo consistentemente nos últimos 30 minutos, isso é uma emergência. Pagine o engenheiro de plantão *agora*."

Consequência: alerta de alta prioridade (PagerDuty, Opsgenie) que exige intervenção humana imediata.

Burn Rate Lento (Slow Burn)

Objetivo: detectar problemas de baixa gravidade, mas persistentes, que, se não forem corrigidos, consumirão lentamente o Error Budget e podem levar à violação do SLO no final da janela (semanas).

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Características:

- **Limiar de Burn Rate:** baixo (ex: 2x, 3x).
- **Janela de Avaliação:** longa (ex: 2 a 6 horas).

Lógica: "Se estamos consistentemente queimando nosso orçamento duas vezes mais rápido que o normal nas últimas 4 horas, isso não é uma emergência imediata, mas não é sustentável. Precisamos que alguém investigue isso durante o horário comercial."

Consequência: alerta de baixa prioridade (ticket no Jira, mensagem no Slack) que pode ser tratado como trabalho planejado.

Monitores avançados

À medida que os sistemas se tornam mais complexos e dinâmicos, os monitores baseados em limiares estáticos, mesmo quando aplicados a SLOs, começam a mostrar suas limitações. Eles são excelentes para detectar quando uma métrica cruza uma fronteira conhecida do "ruim", mas falham em capturar desvios sutis de comportamento, prever problemas futuros ou identificar um único ator malicioso em um cluster de servidores saudáveis.

Para superar esses desafios, o Datadog oferece um conjunto de monitores avançados, alimentados por algoritmos de Machine Learning, que representam a próxima fronteira do alerta inteligente.

Metric Monitor, APM Monitor e Log Monitor

Antes de mergulharmos nos monitores algorítmicos avançados, é fundamental dominar os três tipos de monitores fundamentais que formam a base do sistema de alertas do Datadog. Cada um desses monitores está alinhado com um dos Três Pilares da Observabilidade: Métricas, Traces (APM) e Logs.

Esses monitores são os blocos de construção sobre os quais alertas mais complexos, como os de SLO e os algorítmicos, são frequentemente construídos. Dominá-los permite uma cobertura completa, desde a saúde da infraestrutura subjacente até a lógica de negócio específica registrada em seus logs.

O **Metric Monitor** é o tipo de monitor mais fundamental e versátil do Datadog. Ele permite alertar sobre qualquer métrica numérica que você envia para a plataforma,

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

seja ela uma métrica de infraestrutura, de integração, de APM (embora o APM Monitor seja mais especializado) ou uma métrica customizada.

- **Fonte de Dados:** séries temporais de métricas numéricas.
- **Principal Caso de Uso:** monitorar a saúde da infraestrutura (CPU, memória, disco), métricas de serviços de nuvem (como número de mensagens em uma fila SQS), ou métricas de negócio customizadas (como pedidos.processados ou usuarios.ativos).
- **Força:** extremamente flexível; pode ser usado para qualquer dado numérico.

O **APM Monitor** é um tipo de monitor especializado, projetado para alertar diretamente sobre os dados de alta cardinalidade e ricos em contexto provenientes do tracing de APM. Em vez de usar métricas genéricas, ele opera sobre a performance de serviços, recursos (endpoints), ou até mesmo spans individuais.

- **Fonte de Dados:** traces e spans do APM.
- **Principal Caso de Uso:** alertar sobre a latência (p50, p75, p95, p99), taxa de erro ou taxa de requisições (hits) de um serviço específico ou de um endpoint particular.
- **Força:** oferece um nível de granularidade muito maior para o monitoramento de aplicações, permitindo focar no que realmente importa para a performance do código.

O **Log Monitor** analisa o fluxo de logs ingeridos pelo Datadog em tempo real e dispara alertas com base na contagem de logs que correspondem a uma query específica.

- **Fonte de Dados**: logs.
- **Principal Caso de Uso**: alertar sobre a ocorrência de erros específicos (buscando por uma mensagem de exceção ou stack trace), atividades de segurança suspeitas (como múltiplas tentativas de login falhas), ou uma queda inesperada no volume de logs (o que pode indicar que um serviço parou de logar).

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

- **Força**: permite alertar sobre eventos não-numéricos e de texto, capturando problemas que são impossíveis de detectar apenas com métricas.

Anomaly Detection na prática

O monitor de Anomaly Detection (Detecção de Anomalias) é uma das ferramentas de Machine Learning mais poderosas do Datadog. Sua função é aprender o comportamento "normal" de uma métrica, incluindo suas tendências de longo prazo e padrões sazonais, e então alertar quando a métrica se desvia significativamente desse comportamento esperado. Isso permite capturar problemas que seriam invisíveis para um monitor de limiar estático.

Imagine uma métrica de requisições por segundo para um site de e-commerce. É normal que ela seja alta durante o dia e baixa durante a madrugada. Um limiar estático de "alerta se o tráfego cair abaixo de X" seria inútil, pois geraria alertas falsos todas as noites. O Anomaly Detection, no entanto, aprende esse padrão diário. Ele só alertaria se o tráfego caísse *inesperadamente* às 14h, ou se *não* caísse durante a madrugada, ambos sinais de um problema real.

O algoritmo analisa o histórico da métrica (geralmente várias semanas) e a decompõe em três componentes:

- **Tendência (Trend):** o crescimento ou declínio de longo prazo da métrica.
- **Sazonalidade (Seasonality):** padrões que se repetem em intervalos regulares (horários, diários, semanais).
- **Resíduos (Residuals):** flutuações aleatórias que não são explicadas pela tendência ou sazonalidade.

Com base nesses componentes, o algoritmo cria uma "faixa cinza" (confidence band) em torno do valor esperado da métrica. Se o valor real da métrica sai dessa faixa cinza por um período de tempo, ele é sinalizado como uma anomalia. Algoritmos Disponíveis:

- **Agile:** mais sensível a mudanças recentes no nível da métrica. Bom para métricas que podem mudar seu "normal" rapidamente.
- **Robust:** menos influenciado por picos ou quedas temporárias. Ideal para métricas que têm um comportamento de longo prazo mais estável.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Forecast: previsão de comportamento futuro

Enquanto a detecção de anomalias se concentra em identificar desvios do comportamento *passado*, o monitor de Forecast (Previsão) olha para o futuro. Sua função é analisar a tendência de uma métrica e prever quando ela cruzará um limiar crítico em algum momento futuro. Esta é a forma mais proativa de monitoramento, transformando a equipe de operações de "bombeiros" que apagam incêndios para "planejadores" que evitam que eles comecem.

O caso de uso mais clássico e intuitivo para o forecasting é o monitoramento do uso de disco. Um alerta tradicional de "disco > 90%" é um alerta de pânico; ele informa que o problema já é iminente e requer ação imediata. Um alerta de forecast, por outro lado, pode notificar a equipe com dias ou semanas de antecedência: "Com base na taxa de crescimento atual, o disco do servidor X atingirá 100% em aproximadamente 7 dias." Isso dá à equipe tempo suficiente para planejar e executar uma ação corretiva (como limpar arquivos antigos ou provisionar mais espaço) durante o horário comercial, sem qualquer estresse ou emergência.

O monitor de forecast utiliza algoritmos de regressão para analisar a tendência de uma métrica ao longo do tempo. Ele projeta essa tendência para o futuro, criando uma linha de previsão juntamente com uma faixa de confiança (similar à do monitor de anomalia). O alerta é acionado quando a borda dessa faixa de confiança cruza o limiar definido, dentro de um horizonte de tempo futuro que você especifica. Algoritmos Disponíveis:

- **Linear:** usa uma regressão linear robusta. É ideal para métricas que têm uma tendência de crescimento ou declínio constante, como o uso de disco ou o número de usuários em um banco de dados.
- **Seasonal (Sazonal):** leva em conta padrões sazonais (horários, diários, semanais) além da tendência linear. É útil para métricas de negócio que têm picos e vales previsíveis.

Outlier Detection entre membros de um cluster

Em arquiteturas modernas e distribuídas, os serviços raramente são executados em uma única instância. Eles são implantados como um cluster de servidores, containers ou funções serverless, todos trabalhando em conjunto para atender à demanda. Nesse cenário, um desafio comum de monitoramento é identificar

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

quando *um único membro* do cluster começa a se comportar de maneira diferente dos seus pares.

Este é o problema do "vizinho barulhento" ou da "maçã podre". Um único servidor com um problema de hardware sutil, uma configuração incorreta ou um bug de contenção de thread pode ter sua performance degradada, mas se o tráfego for distribuído entre muitos outros nós saudáveis, a média geral da métrica (como a latência) pode parecer perfeitamente normal. O problema permanece oculto até que se agrave ou que mais nós comecem a falhar.

O monitor de Outlier Detection (Detecção de Outliers) do Datadog foi projetado especificamente para resolver este problema. Ele não compara uma métrica com um limiar estático ou com seu próprio histórico (como o Anomaly Detection), mas sim compara o comportamento de um membro de um grupo com o comportamento de *todos os outros membros do mesmo grupo*, em tempo real.

O algoritmo agrupa as séries temporais de uma métrica por uma dimensão (por exemplo, host ou container_id). Em seguida, ele calcula a média ou a mediana do grupo e identifica qualquer membro que se desvie dessa linha de base do grupo por uma certa quantidade. Ele é particularmente eficaz em encontrar:

- **Um host lento em um cluster:** todos os servidores web .NET em um cluster devem ter uma latência p95 semelhante. Se um deles de repente se torna 2x mais lento, o Outlier Detection irá sinalizá-lo.
- **Um consumidor de fila lento:** em um grupo de workers processando mensagens de uma fila, se um deles começa a processar mensagens a uma taxa muito mais baixa, ele é um outlier.
- **Desbalanceamento de carga:** se um servidor em um cluster está recebendo significativamente mais ou menos tráfego que os outros, isso pode indicar um problema com o load balancer.

Algoritmos Disponíveis:

- **DBSCAN:** um algoritmo de clustering que é robusto e não assume que os dados seguem uma distribuição normal. É o padrão e geralmente a melhor escolha.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

- **MAD (Median Absolute Deviation):** mais rápido que o DBSCAN, mas assume que os dados sem outliers seguem uma distribuição normal. Pode ser útil para grupos muito grandes.

Monitores de disponibilidade

Além dos monitores que analisam o comportamento de métricas, logs e traces, existe uma classe de monitores cujo único propósito é responder a uma pergunta binária e fundamental: "Este sistema ou endpoint está online e respondendo?" Esses são os monitores de disponibilidade. Eles não medem a performance, mas sim a própria existência e acessibilidade de um componente. São a primeira linha de defesa e a forma mais básica de verificação de saúde de um sistema.

O Datadog oferece vários tipos de monitores de disponibilidade, cada um projetado para verificar um aspecto diferente do seu ecossistema. Três dos mais importantes são o Host Monitor, o Integration Monitor e o Synthetic Test.

Host Monitor é o monitor de disponibilidade mais simples. Ele verifica se o Datadog Agent, instalado em um dos seus servidores, está reportando métricas. Se o Agent parar de enviar dados por um determinado período, o monitor dispara um alerta.

- **Pergunta que Responde:** "O servidor está online e o Datadog Agent está funcionando?"
- **Principal Caso de Uso:** detectar quando um servidor inteiro fica offline, seja por uma falha de hardware, um problema de rede ou uma parada inesperada do sistema operacional.
- **Força:** é um sinal de "pulsação" fundamental. Se um host para de reportar, todos os serviços que rodam nele estão, por definição, em risco.

Integration Monitor verifica a saúde das integrações do Datadog com serviços de terceiros, como AWS, Azure, GCP, ou até mesmo aplicações como PostgreSQL e NGINX. Cada integração reporta um *service check* que indica seu status (OK, WARNING, CRITICAL).

- **Pergunta que Responde:** "A integração do Datadog com o meu serviço de nuvem ou aplicação está funcionando corretamente?"

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

- **Principal Caso de Uso:** detectar problemas de permissão (ex: a role do IAM da AWS expirou), falhas na API do provedor de nuvem, ou problemas de conectividade entre o Agent e a aplicação que ele está monitorando.

O **Synthetic Test** é o monitor de disponibilidade mais poderoso e representativo da experiência do usuário. Ele simula ativamente as jornadas do usuário em sua aplicação, de fora para dentro, exatamente como um cliente faria. Ele pode fazer desde um simples ping em um endpoint (API Test) até uma complexa transação de múltiplos passos em um site (Browser Test).

Pergunta que Responde: "Um usuário real consegue acessar e usar minha aplicação de um local específico do mundo?"

Principal Caso de Uso:

- **API Test:** verificar a disponibilidade, correção e performance de um endpoint de API. Pode validar o status code da resposta, o conteúdo do corpo e os headers.
- **Browser Test:** simular uma jornada completa do usuário, como login, busca de um produto, adição ao carrinho e checkout, alertando se qualquer passo falhar.

Força: é o único tipo de monitor que pode detectar problemas *antes* que os usuários reais os encontrem. Ele funciona mesmo que não haja tráfego real na aplicação, tornando-o essencial para verificar a saúde do sistema após um deploy noturno ou durante períodos de baixa atividade.

Error Tracking para agrupamento inteligente de erros

Em qualquer aplicação complexa, erros são inevitáveis. Uma única falha pode gerar milhares de entradas de log de erro em um curto período, criando um dilúvio de dados que é impossível de analisar manualmente. Tradicionalmente, os desenvolvedores vasculhavam arquivos de log ou streams de logs em busca de exceções, uma tarefa reativa, demorada e ineficiente.

Cada linha de log de erro era tratada como um evento individual, tornando impossível distinguir entre dez mil ocorrências de um único bug e dez mil bugs diferentes ocorrendo uma única vez. O **Error Tracking** do Datadog surge como uma

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

solução para este problema, transformando o caos de logs de erro em um fluxo de trabalho de gerenciamento de issues organizado e acionável.

Agrupamento automático de erros

A capacidade do Error Tracking de transformar um fluxo caótico de erros em uma lista gerenciável de Issues depende inteiramente de seus algoritmos de **agrupamento automático**. O objetivo desses algoritmos é analisar cada erro que chega e decidir se ele é uma nova ocorrência de uma Issue já existente ou se representa um bug completamente novo. Essa decisão é baseada em um conjunto de heurísticas sofisticadas que geram uma "impressão digital" ou **fingerprint** para cada erro.

Erros com a mesma fingerprint são agrupados na mesma Issue. Essa fingerprint é projetada para ser específica o suficiente para distinguir entre bugs diferentes, mas geral o suficiente para ignorar variações irrelevantes (como timestamps, IDs de requisição ou nomes de hosts específicos) que fariam com que cada erro parecesse único.

O Datadog utiliza uma cascata de heurísticas para gerar a fingerprint de um erro, sendo a mais importante a análise do **stack trace** (pilha de chamadas).

O stack trace é a fonte de informação mais rica para identificar a origem de um erro. Ele mostra a sequência exata de chamadas de método que levaram à exceção. O algoritmo de agrupamento do Datadog analisa o stack trace para extrair as informações mais relevantes.

Tipo da Exceção (Exception Type): o nome da classe da exceção é um dos principais componentes da fingerprint. Uma System.NullReferenceException é fundamentalmente diferente de uma System.Data.SqlClient.SqlException.

Mensagem da Exceção (Exception Message): a mensagem pode conter detalhes importantes, mas o algoritmo é inteligente o suficiente para remover partes variáveis (como IDs ou valores específicos) e focar na estrutura da mensagem.

Frames da Pilha (Stack Frames): o algoritmo examina os frames do stack trace, focando nos frames que pertencem ao código da sua aplicação (e geralmente ignorando os frames do framework, como os do próprio .NET Core). O nome do

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

arquivo, o nome do método e, às vezes, o número da linha onde a exceção ocorreu são os elementos mais críticos.

Às vezes, o stack trace por si só não é suficiente, ou os erros podem não ter um stack trace (por exemplo, erros de log simples). Nesses casos, o Datadog recorre a outras heurísticas:

Proximidade no Código-Fonte: para erros que ocorrem em arquivos de log, o algoritmo pode considerar a mensagem de log e o nome do arquivo de origem (se disponível) para agrupar erros semelhantes.

Atributos do Erro: tags e atributos associados ao erro, como http.method e http.url, podem ser usados como um critério secundário para agrupar erros que ocorrem no mesmo contexto de requisição.

Em casos avançados, as heurísticas padrão podem não ser ideais. Por exemplo, você pode ter um erro genérico de "Falha na Transação" onde a verdadeira causa raiz está em um código de erro interno. O Datadog permite que você defina **regras de fingerprinting personalizadas** para sobrescrever o comportamento padrão. Você pode instruir o Datadog a usar uma tag ou um atributo específico do log (como transaction.error_code) como o principal componente da fingerprint, garantindo que erros com o mesmo código de erro interno sejam agrupados, independentemente de pequenas variações no stack trace.

Ciclo de vida de uma Issue

O agrupamento automático de erros é apenas o começo da jornada. O verdadeiro poder do Error Tracking do Datadog reside em fornecer um fluxo de trabalho estruturado para gerenciar cada Issue, desde sua primeira aparição até sua resolução final. Este fluxo de trabalho, conhecido como o ciclo de vida da Issue, transforma o Error Tracking de uma simples ferramenta de visualização em um sistema de gerenciamento de incidentes de bugs, permitindo que as equipes colaborem, rastreiem o progresso e garantam que os problemas sejam resolvidos de forma eficiente.

Cada Issue no Datadog passa por uma série de estágios, refletindo seu status no processo de resolução. Compreender e utilizar esses estágios é fundamental para uma gestão de erros eficaz. O ciclo de vida de uma Issue pode ser visualizado como um funil, onde os erros são detectados, priorizados e, finalmente, resolvidos.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Detecção e Agrupamento (Detection & Grouping)

- **O que acontece**: esta é a fase inicial e automatizada. O Datadog APM Tracer ou a ingestão de logs captura um erro de sua aplicação .NET. Os algoritmos de agrupamento (discutidos no subtópico anterior) analisam o erro e geram uma fingerprint.
- **Resultado**: o erro é atribuído a uma Issue existente ou, se a fingerprint for nova, uma nova Issue é criada. A nova Issue entra no estado Unresolved (Não Resolvida) e no sub-estado New (Nova).

Priorização (Prioritization)

- **O que acontece**: a equipe de desenvolvimento ou SRE analisa a lista de Issues não resolvidas. A UI do Error Tracking fornece dados cruciais para a priorização:
  - Count: quantas vezes o erro ocorreu?
  - Users Affected: quantos usuários únicos foram impactados?
  - First Seen / Last Seen: o erro é novo ou recorrente?
  - Service / Version: qual serviço e qual versão do código estão gerando o erro?
- **Resultado**: com base nesses dados, a equipe decide a urgência da Issue. Um erro que afeta muitos usuários ou uma funcionalidade crítica de negócio (como o checkout) recebe alta prioridade.

Investigação (Investigation)

- **O que acontece**: um desenvolvedor seleciona uma Issue de alta prioridade para investigar. A página de detalhes da Issue é o centro de comando para a depuração:
  - Stack Trace: o stack trace completo e limpo aponta para a origem do erro no código.
  - Traces: a seção mais poderosa. Para cada ocorrência do erro, o Datadog fornece um link para o trace distribuído completo da requisição que causou o erro.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

- Logs: logs relevantes da mesma requisição são automaticamente correlacionados e exibidos.
- Tags: tags como host, container_id, user.id fornecem contexto adicional.

- **Resultado**: o desenvolvedor formula uma hipótese sobre a causa raiz do bug.

Atribuição (Assignment)

- **O que acontece**: uma vez que a Issue é compreendida, ela precisa de um dono. A Issue pode ser atribuída a um desenvolvedor específico ou a uma equipe dentro do Datadog. Além disso, integrações com ferramentas como Jira, Slack ou Azure DevOps permitem que a Issue seja formalmente registrada no sistema de gerenciamento de projetos da equipe.
- **Resultado**: a responsabilidade pela correção do bug é claramente definida. A Issue no Datadog pode ser movida para o estado Assigned (Atribuída).

Resolução (Resolution)

- **O que acontece**: o desenvolvedor escreve o código para corrigir o bug, submete um pull request e a correção é implantada em produção.
- **Resultado**: após o deploy da correção, o Datadog continua monitorando. Se a fingerprint da Issue não for mais detectada por um período de tempo, o Datadog pode automaticamente marcar a Issue como Resolved (Resolvida). Alternativamente, o desenvolvedor pode marcá-la manualmente.

Traces dentro do Error Tracking

Se o agrupamento de erros é o que torna o Error Tracking gerenciável, a integração profunda com o tracing distribuído (APM) é o que o torna acionável. Um stack trace informa *onde* no código um erro ocorreu, mas um trace distribuído informa o *porquê*. Ele fornece a narrativa completa da requisição que levou ao erro, desde o momento em que ela chegou ao seu sistema até o ponto da falha. Esta união de erro e trace é a funcionalidade mais poderosa para acelerar a depuração e a análise de causa raiz.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Para cada ocorrência de um erro agrupado em uma Issue, o Datadog Error Tracking não apenas armazena o erro em si, mas também captura e vincula o trace completo da transação que o gerou. Isso significa que, com um único clique, um desenvolvedor pode passar de uma visão agregada de um bug para uma visão microscópica de uma instância específica e falha daquele bug.

Ao selecionar uma amostra de erro dentro de uma Issue e navegar para o trace associado, um desenvolvedor de .NET obtém um tesouro de informações contextuais que são cruciais para a depuração:

Visualização em Flame Graph: o trace é apresentado como um flame graph, mostrando a hierarquia de chamadas (spans) e a duração de cada uma. O span exato onde a exceção ocorreu é destacado em vermelho, permitindo a localização imediata do ponto de falha no fluxo da requisição.

Tags e Metadados da Requisição: o span raiz do trace (geralmente o que representa a requisição HTTP de entrada) contém tags essenciais.

Interações com Dependências: o trace mostra todas as chamadas de saída da sua aplicação .NET para outros serviços.

Logs Correlacionados: o Datadog correlaciona automaticamente os logs gerados durante a execução daquela requisição específica ao trace.

Configuração de notificações e template variables

Um sistema de monitoramento, por mais inteligente que seja, é ineficaz se seus alertas não chegarem às pessoas certas, no momento certo e com o contexto necessário para uma ação rápida. A configuração de notificações é a última milha crítica no ciclo de vida de um alerta; é a ponte que conecta a detecção de um problema no Datadog com a intervenção humana ou automatizada que irá resolvê-lo. Uma notificação mal configurada é apenas ruído; uma notificação bem elaborada é um chamado à ação, um mini-runbook entregue diretamente nas ferramentas que as equipes já utilizam.

O objetivo é transformar uma notificação genérica como "CPU alta no host X" em uma mensagem rica e acionável como:

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

ALERTA CRÍTICO: A latência p99 do serviço pagamentos-api (versão 1.2.5) está em 850ms, violando o SLO de 500ms. O orçamento de erro está sendo consumido a 10x a taxa normal. O host prod-payments-04 é um outlier de latência. Runbook de investigação: [link]

Essa notificação inteligente não apenas informa sobre o problema (sintoma), mas também fornece contexto crucial: qual serviço, qual versão, qual SLO, qual host específico e, o mais importante, o que fazer a respeito. Para uma equipe de desenvolvimento .NET, receber um alerta que já identifica o serviço e a versão do código afetados significa que eles podem começar a investigar a mudança de código relevante imediatamente.

Integração completa com Slack

No ambiente de trabalho moderno, o Slack transcendeu sua função de ferramenta de chat para se tornar o centro nevrálgico da colaboração técnica. É onde as equipes discutem problemas, compartilham informações e coordenam respostas a incidentes. Portanto, integrar o sistema de alertas do Datadog diretamente ao Slack não é apenas uma conveniência, mas uma etapa fundamental para criar um fluxo de trabalho de resposta a incidentes coeso e eficiente. Enviar alertas para o Slack significa entregar a informação diretamente no local onde a conversa e a ação já estão acontecendo.

Uma integração bem-sucedida transforma um canal do Slack em uma sala de guerra (war room) dinâmica, onde os alertas do Datadog fornecem o contexto inicial, e a equipe pode imediatamente colaborar na investigação e resolução, tudo em um único local, com um histórico pesquisável.

Por que integrar com Slack?

- **Visibilidade Imediata:** alertas são vistos instantaneamente pela equipe relevante no canal apropriado, em vez de se perderem em uma caixa de entrada de e-mail.
- **Contexto Compartilhado:** toda a equipe no canal vê o mesmo alerta, garantindo que todos estejam na mesma página. Isso elimina a necessidade de encaminhar e-mails ou copiar e colar mensagens de erro.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

- **Colaboração em Tempo Real:** as discussões sobre o alerta podem começar imediatamente em uma thread do Slack, mantendo a conversa organizada e vinculada ao evento original.
- **Ações Rápidas:** a partir de um alerta no Slack, as equipes podem iniciar ações, como criar um ticket no Jira, iniciar uma vídeo chamada ou acionar um workflow automatizado, tudo a partir da mesma interface.

Variáveis avançadas de template

As **Template Variables (variáveis de template)** são o segredo para transformar notificações genéricas em mensagens ricas e específicas ao contexto. Elas são placeholders, como {{host.name}} ou {{value}}, que o Datadog substitui dinamicamente pelos valores reais do alerta no momento em que a notificação é enviada. O uso eficaz dessas variáveis é o que permite que uma única mensagem de monitor sirva para centenas de hosts ou serviços, fornecendo sempre os detalhes exatos do recurso que está em alerta.

Dominar o vocabulário das variáveis de template é essencial para qualquer engenheiro que queira criar um sistema de alertas verdadeiramente informativo. Elas são a ponte entre os dados brutos do monitor e a mensagem legível por humanos que chega no Slack ou no PagerDuty.

O Datadog oferece uma vasta gama de variáveis, que podem ser agrupadas em várias categorias.

Variáveis de Valor e Limiar: estas são as variáveis mais básicas e essenciais. Elas descrevem o estado numérico do monitor.

Variável	Descrição	Exemplo de Uso
`{{value}}`	O valor da métrica no momento do alerta.	`A latência atual é {{value}}ms.`
`{{threshold}}`	O limiar que foi cruzado para disparar o alerta.	`...excedendo o limiar de {{threshold}}ms.`
`{{warn_threshold}}`	O limiar de aviso (se aplicável).	`O limiar de aviso é {{warn_threshold}}.`
`{{critical_threshold}}`	O limiar crítico (se aplicável).	`O limiar crítico é {{critical_threshold}}.`

Figura 5 – Variáveis de valor e limiar
Fonte: Elaborado pelo autor (2026)

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Variáveis de Tags e Agregação: quando um monitor é agrupado (por exemplo, group by host), estas variáveis permitem acessar as tags do grupo específico que está em alerta. Elas são cruciais para identificar *onde* o problema está acontecendo.

Variável	Descrição	Exemplo de Uso
`{{host.name}}`	O nome do host que está em alerta.	`Alerta no host: {{host.name}}`
`{{service.name}}`	O nome do serviço (da tag `service`).	`Serviço afetado: {{service.name}}`
`{{<TAG_KEY>.name}}`	O valor da tag para a chave `<TAG_KEY>`.	`Região: {{region.name}}`
`{{<TAG_KEY>.value}}`	Sinônimo de `{{<TAG_KEY>.name}}`.	`Ambiente: {{env.value}}`

Figura 6 – Variáveis de Tags e Agregação
Fonte: Elaborado pelo autor (2026)

Variáveis de SLO (para Monitores de SLO): monitores de SLO têm seu próprio conjunto de variáveis que fornecem um contexto rico sobre o estado do SLO e do Error Budget.

Variável	Descrição	Exemplo de Uso
`{{slo.name}}`	O nome do SLO.	`Alerta no SLO: {{slo.name}}`
`{{slo.id}}`	O ID alfanumérico do SLO.	`Link para o SLO: https://app.datadoghq.com/slo?id={{slo.id}}`
`{{slo.error_budget_remaining}}`	A porcentagem do orçamento de erro restante.	`Orçamento de Erro Restante: {{slo.error_budget_remaining}}%`
`{{slo.target}}`	A meta percentual do SLO.	`Meta do SLO: {{slo.target}}%`
`{{time_window}}`	A janela de tempo do SLO (ex: `7d`).	`Janela de Tempo: {{time_window}}`

Figura 7 – Variáveis de SLO
Fonte: Elaborado pelo autor (2026)

Variáveis de Informação do Monitor: estas variáveis fornecem metadados sobre o próprio monitor.

Variável	Descrição	Exemplo de Uso
`{{monitor.name}}`	O nome do monitor.	`Monitor: {{monitor.name}}`
`{{monitor.id}}`	O ID do monitor.	`Link para o monitor: https://app.datadoghq.com/monitors/{{monitor.id}}`
`{{monitor.status}}`	O estado atual do monitor (`ALERT`, `WARN`, `OK`).	`Status atual: {{monitor.status}}`

Figura 8 – Variáveis de Informação do Monitor
Fonte: Elaborado pelo autor (2026)

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Blocos condicionais na mensagem

As variáveis de template permitem que as notificações sejam dinâmicas em relação aos *dados* do alerta. Os Blocos Condicionais levam essa dinamicidade um passo adiante, permitindo que a própria *estrutura* da notificação mude com base no *estado* do alerta. Eles são essencialmente declarações if/else para suas mensagens de notificação, permitindo que você mostre ou oculte seções inteiras da mensagem dependendo se o monitor está em estado de ALERT, WARN, OK (recuperado), ou NO DATA.

O uso de blocos condicionais é a chave para criar um sistema de notificação limpo e eficiente, que comunica claramente não apenas o início de um problema, mas também sua resolução, usando o mesmo monitor e a mesma configuração de notificação.

A sintaxe é simples e se assemelha a muitos motores de template:

- {{#is_alert}} ... {{/is_alert}}: o conteúdo dentro deste bloco só será exibido se o monitor estiver no estado ALERT.
- {{#is_warn}} ... {{/is_warn}}: o conteúdo só será exibido se o monitor estiver no estado WARN.
- {{#is_recovery}} ... {{/is_recovery}}: o conteúdo só será exibido quando o monitor se recuperar (voltar para o estado OK).
- {{#is_no_data}} ... {{/is_no_data}}: Oo conteúdo só será exibido se o monitor entrar no estado NO DATA.

Além desses, existem blocos de negação:

- {{^is_alert}} ... {{/is_alert}}: o conteúdo será exibido se o estado *não* for ALERT.

E um bloco else genérico:

- {{else}} ...: usado dentro de um bloco principal (como {{#is_alert}}) para fornecer conteúdo alternativo.

Por que usar blocos condicionais?

Notificações de Recuperação Claras: a notificação de recuperação é tão importante quanto a de alerta. Ela informa à equipe que o problema foi resolvido e que

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

eles podem parar a investigação. Usar um bloco {{#is_recovery}} permite criar uma mensagem de recuperação completamente diferente e tranquilizadora.

Redução de Ruído: em vez de ter uma mensagem de alerta que inclui texto como "Se este for um alerta de recuperação...", você pode separar completamente as mensagens, tornando cada notificação mais concisa e focada em seu propósito.

Instruções Específicas ao Estado: a ação a ser tomada para um alerta crítico é diferente da de um aviso. Você pode usar {{#is_alert}} para incluir um link para o PagerDuty e @-mencionar a equipe de plantão, enquanto no bloco {{#is_warn}} você pode apenas criar um ticket no Jira.

Criando notificações ricas e acionáveis

Chegamos ao ponto culminante da nossa jornada pela configuração de notificações: a síntese de todos os elementos que aprendemos - integrações, variáveis de template e blocos condicionais - para construir a notificação "perfeita". Uma notificação rica e acionável não é apenas uma mensagem; é um pacote de informações curado, projetado para minimizar o "tempo para o contexto".

O objetivo é fornecer ao engenheiro de plantão informações suficientes para que ele possa, em segundos, entender a natureza do problema, sua gravidade, seu impacto e os próximos passos para a resolução, tudo isso sem precisar sair da ferramenta de comunicação (como o Slack).

Uma notificação de alta qualidade é a diferença entre uma resposta a incidentes caótica e uma resposta calma e metódica. Ela reduz a carga cognitiva do respondente e o orienta para a solução.

Uma notificação acionável deve responder a cinco perguntas fundamentais:

1. O quê? O que exatamente está quebrado? Qual métrica cruzou qual limiar?
2. Onde? Qual serviço, host, container, endpoint ou região geográfica está sendo afetado?
3. Qual a Gravidade? Isso é uma emergência que requer ação imediata ou um aviso que pode ser tratado depois? Qual é o impacto no usuário ou no SLO?
4. Qual o Contexto? Qual a tendência? O problema está piorando? O que mais aconteceu nesse período (deploys, outros alertas)?

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

5. E Agora? O que eu devo fazer a respeito? Onde estão os dashboards, runbooks e ferramentas de comunicação relevantes?

A tabela a seguir mapeia essas perguntas para as funcionalidades do Datadog que usamos para respondê-las:

Pergunta	Funcionalidade Datadog	Exemplo na Notificação
O quê?	Nome do Monitor, `{{value}}`, `{{threshold}}`	`Latência p99 (850ms) excedeu o limiar (500ms).`
Onde?	Tags (`{{service.name}}`, `{{host.name}}`, etc.)	`Serviço: pagamentos-api, Host: prod-payments-04`
Qual a Gravidade?	Blocos Condicionais, `{{monitor.status}}`, `{{slo.error_budget_remaining}}`	🚨 `CRÍTICO` 🚨, `Orçamento de Erro Restante: 25%`
Qual o Contexto?	Snapshot do Gráfico, `{{version.name}}`	Imagem do gráfico de latência, `Versão: 1.3.1`
E Agora?	Links para Dashboards, Runbooks, Menções (`@slack`, `@pagerduty`)	`Runbook: [link]`, `@pagerduty-oncall`

Figura 9 – funcionalidades do Datadog
Fonte: Elaborado pelo autor (2026)

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, você compreendeu o papel estratégico dos dashboards na observabilidade, aprendendo a aplicar princípios de hierarquia visual e densidade de informação para uma análise eficiente. A partir de agora você será capaz de escolher o widget ideal para cada questão operacional, padronizando elementos como cores e eixos, além de utilizar a correlação visual e a contextualização de métricas com eventos para acelerar processos de troubleshooting.

Além disso, focamos na gestão de confiabilidade, onde você aprendeu a diferenciar e aplicar os conceitos de SLI, SLO e Error Budget. O aprendizado incluiu o cálculo de orçamentos de erro baseados em tempo ou eventos, a interpretação de seu consumo e o domínio do conceito de *burn rate*. Neste momento, você está apto(a) a configurar alertas de queima rápida e lenta, priorizando notificações baseadas em sintomas reais para uma operação mais resiliente.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

REFERÊNCIAS

DATADOG. **Dashboards Best Practices**. Disponível em: https://docs.datadoghq.com/dashboards/best_practices/. Acesso em: 09 jan. 2026.

DATADOG. **Service Level Objectives (SLOs).** Disponível em: https://docs.datadoghq.com/service_management/service_level_objectives/. Acesso em: 09 jan. 2026.

GOOGLE. **Monitoring Distributed Systems**. Disponível em: https://sre.google/sre-book/monitoring-distributed-systems/. Acesso em: 09 jan. 2026.

GOOGLE. **Site Reliability Engineering: How Google Runs Production Systems**. Sebastopol: O'Reilly Media, 2016.

GOOGLE. **The Site Reliability Workbook. Sebastopol**: O'Reilly Media, 2018.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

PALAVRAS-CHAVE

Observabilidade, Dashboards, SLO

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com



PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com